

React Native Core Components

This guide focuses on the essential core components in React Native for building mobile applications.

Prerequisites: Complete the Environment Setup Guide and create your first app.

Reference: <https://reactnative.dev/docs/getting-started>

Introduction

React Native is a framework for building native mobile applications using JavaScript and React. When you create an Expo app, you'll work with several key files that control your app's configuration and structure.

app.json - Application Configuration

The `app.json` file is a configuration file that defines your app's metadata and settings for Expo. It contains important information about your app:

Structure:

```
{
  "expo": {
    "name": "my-expo-app",          // Display name of your app
    "slug": "my-expo-app",         // URL-friendly identifier
    "version": "1.0.0",            // App version number
    "orientation": "portrait",    // Screen orientation (portrait/landscape)
    "icon": "./assets/icon.png",   // App icon path
    "userInterfaceStyle": "light", // Theme preference
    "splash": {                   // Splash screen settings
      "image": "./assets/splash-icon.png",
      "resizeMode": "contain",
      "backgroundColor": "#ffffff"
    },
    "ios": {                       // iOS-specific settings
      "supportsTablet": true
    },
    "android": {                   // Android-specific settings
      "adaptiveIcon": {
        "foregroundImage": "./assets/adaptive-icon.png",
        "backgroundColor": "#ffffff"
      }
    },
    "web": {                       // Web-specific settings
      "favicon": "./assets/favicon.png"
    }
  }
}
```

Key Points:

- This file configures your app's appearance and behavior
 - Platform-specific settings (iOS, Android, Web) are nested under their respective keys
 - Asset paths are relative to the project root
 - Changes to this file may require restarting the development server
-

JSX (JavaScript XML) - Structure and Notes

JSX is a syntax extension that allows you to write HTML-like code in JavaScript. It's not HTML, but looks similar.

JSX Structure:

1. **Single Parent Element:** JSX must return a single parent element

```
// ☒ Valid - single parent
return (
  <View>
    <Text>Hello</Text>
    <Text>World</Text>
  </View>
);

// ☐ Invalid - multiple root elements
return (
  <Text>Hello</Text>
  <Text>World</Text>
);
```

2. **Self-Closing Tags:** All tags must be closed

```
// ☒ Valid
<View />
<TextInput />

// ☐ Invalid
<View>
<TextInput>
```

3. **JavaScript Expressions:** Use curly braces `{}` to embed JavaScript

```
const name = "John";
return <Text>Hello, {name}!</Text>;
```

4. **Attributes (Props):** Use camelCase for attributes

```
<Text style={{ fontSize: 18 }}>Hello</Text>
<View style={styles.container} />
```

5. **Comments:** Use `{/* */}` syntax inside JSX

```
return (
  <View>
    {/* This is a comment */}
    <Text>Hello</Text>
  </View>
);
```

Key JSX Rules:

- Use **camelCase** for attributes (e.g., `onClick`, `backgroundColor`)
- All tags must be **closed** (self-closing with `/`)
- JavaScript expressions go inside `{}`
- Class becomes `className` (though not used in React Native)
- Must return **one parent element** (or use React Fragment `<>...</>`)

App.js - Structure Explanation

When you run `npx create-expo-app`, it generates a simple default `App.js` file. This is the main entry point for your React Native application. Let's break down the default generated structure:

Default Generated App.js Code:

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>Open up App.js to start working on your app!</Text>
      <StatusBar style="auto" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
```

```
    justifyContent: 'center',  
  },  
});
```

Structure Breakdown:

1. Import Statements

```
import { StatusBar } from 'expo-status-bar';  
import { StyleSheet, Text, View } from 'react-native';
```

- **StatusBar** from **expo-status-bar**: Controls the device status bar (battery, time, signal icons)
- **StyleSheet** from **react-native**: Creates optimized style objects
- **Text** from **react-native**: Component for displaying text
- **View** from **react-native**: Container component (like `<div>` in HTML)

2. Function Component Declaration

```
export default function App() {  
  // Component code here  
}
```

- **App**: The name of the main component
- **export default**: Makes this component the default export (entry point)
- **function App()**: Declares a function component that returns JSX
- This is the root component that React Native renders

3. Return Statement (JSX)

```
return (  
  <View style={styles.container}>  
    <Text>Open up App.js to start working on your app!</Text>  
    <StatusBar style="auto" />  
  </View>  
);
```

- **Returns JSX** that defines what to display on screen
- **Single parent element**: The `<View>` wraps all child components
- `<Text>`: Displays the welcome message
- `<StatusBar>`: Sets the status bar style (auto-detects light/dark mode)

4. StyleSheet Definition

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

- **StyleSheet.create():** Creates optimized style objects (better performance than inline styles)
- **styles.container:** A named style object for the container View
- **Style properties explained:**
 - **flex: 1:** Takes up all available space (fills the screen)
 - **backgroundColor: '#fff':** Sets white background color
 - **alignItems: 'center':** Centers children horizontally (cross-axis)
 - **justifyContent: 'center':** Centers children vertically (main-axis)

Key Concepts:

- **Function Component:** A JavaScript function that returns JSX to render UI
- **JSX:** HTML-like syntax that describes the UI structure
- **Props:** Properties passed to components (e.g., `style={styles.container}`)
- **StyleSheet:** Optimized way to define styles (located at the bottom of the file)
- **Default Export:** The component that serves as the app's entry point

Core Components

React Native provides built-in components that map to native platform components:

View

Container component (like `<div>` in web)

- Used to wrap and group other components
- Supports flexbox layout by default
- Can be styled with StyleSheet

Example:

```
import { View } from 'react-native';

<View style={{ padding: 20 }}>
  {/* Your content here */}
</View>
```

Text

Displays text (like `<p>` in web)

- Required for displaying any text in React Native
- Text must be wrapped in `<Text>` component
- Supports nested text with different styles

Example:

```
import { Text } from 'react-native';

<Text style={{ fontSize: 18, color: '#333' }}>
  Hello, React Native!
</Text>
```

Image

Displays images

- Can load images from local assets or remote URLs
- Requires explicit width and height (or use `resizeMode`)

Example:

```
import { Image } from 'react-native';

// Local image
<Image source={require('./assets/icon.png')} />

// Remote image
<Image
  source={{ uri: 'https://example.com/image.jpg' }}
  style={{ width: 200, height: 200 }}
/>
```

ScrollView

Scrollable container

- Allows content to scroll when it exceeds screen size
- Use for small lists or scrollable content
- For long lists, prefer `FlatList` for better performance

Example:

```
import { ScrollView, Text } from 'react-native';

<ScrollView>
```

```
<Text>Scrollable content here...</Text>
</ScrollView>
```

TextInput

Text input field

- Allows users to enter text
- Controlled component (use `value` and `onChangeText`)
- Supports various keyboard types and input modes

Example:

```
import { useState } from 'react';
import { TextInput } from 'react-native';

function MyInput() {
  const [text, setText] = useState('');

  return (
    <TextInput
      value={text}
      onChangeText={setText}
      placeholder="Enter text..."
      style={{ borderWidth: 1, padding: 10 }}
    />
  );
}
```

Button

Pressable button

- Simple button component with `title` and `onPress`
- Limited styling options (use `TouchableOpacity` for custom styling)

Example:

```
import { Button } from 'react-native';

<Button
  title="Click Me"
  onPress={() => alert('Button pressed!')}
/>
```

StyleSheet

Style definitions

- Used to create style objects for components
- Better performance than inline styles
- Similar to CSS but uses camelCase and JavaScript objects

Example:

```
import { StyleSheet, View, Text } from 'react-native';

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#fff',
  },
  text: {
    fontSize: 20,
    color: '#333',
    fontWeight: 'bold',
  },
});

function MyComponent() {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>Styled Text</Text>
    </View>
  );
}
```

Key Styling Notes:

- Use camelCase (e.g., `backgroundColor` not `background-color`)
- No units needed for dimensions (numbers are density-independent pixels)
- Flexbox is the default layout system

Resources

- **React Native Documentation:** <https://reactnative.dev/docs/getting-started>
- **React Native Components:** <https://reactnative.dev/docs/components-and-apis>
- **Expo Documentation:** <https://docs.expo.dev>